

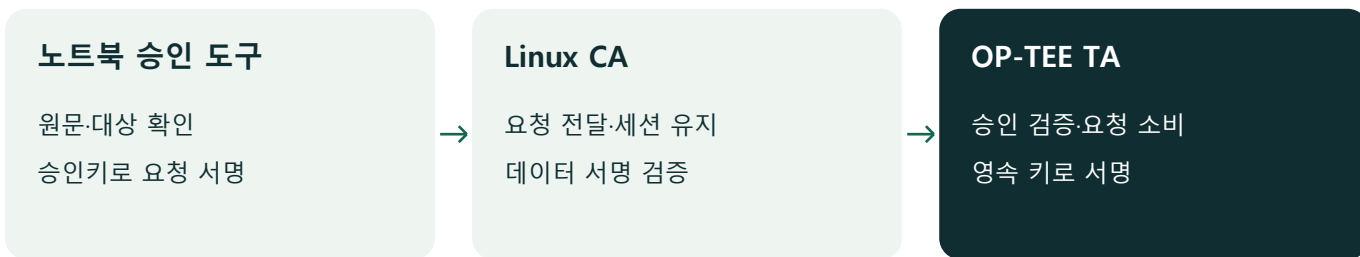
승인된 요청만 서명하는 OP-TEE 서비스

STM32MP157F-DK2 · OpenSTLinux / Yocto · CA/TA 실증

개인키를 반환하지 않는 것만으로는 무단 서명 요청을 막을 수 없다. 외부 승인자가 허가한 요청을 TA가 직접 검증하고, 세션에 연결된 데이터에만 서명하도록 구현했다.

실증 결과	확인 범위
21개 실행·확인 항목 PASS	기초 기능 · 승인 정상/거부 · 재부팅 키 유지
실물 보드 / SSH 수집	2026-10-02 · portfolio-hello 1.0-r0.10

두 키의 역할과 실행 경계



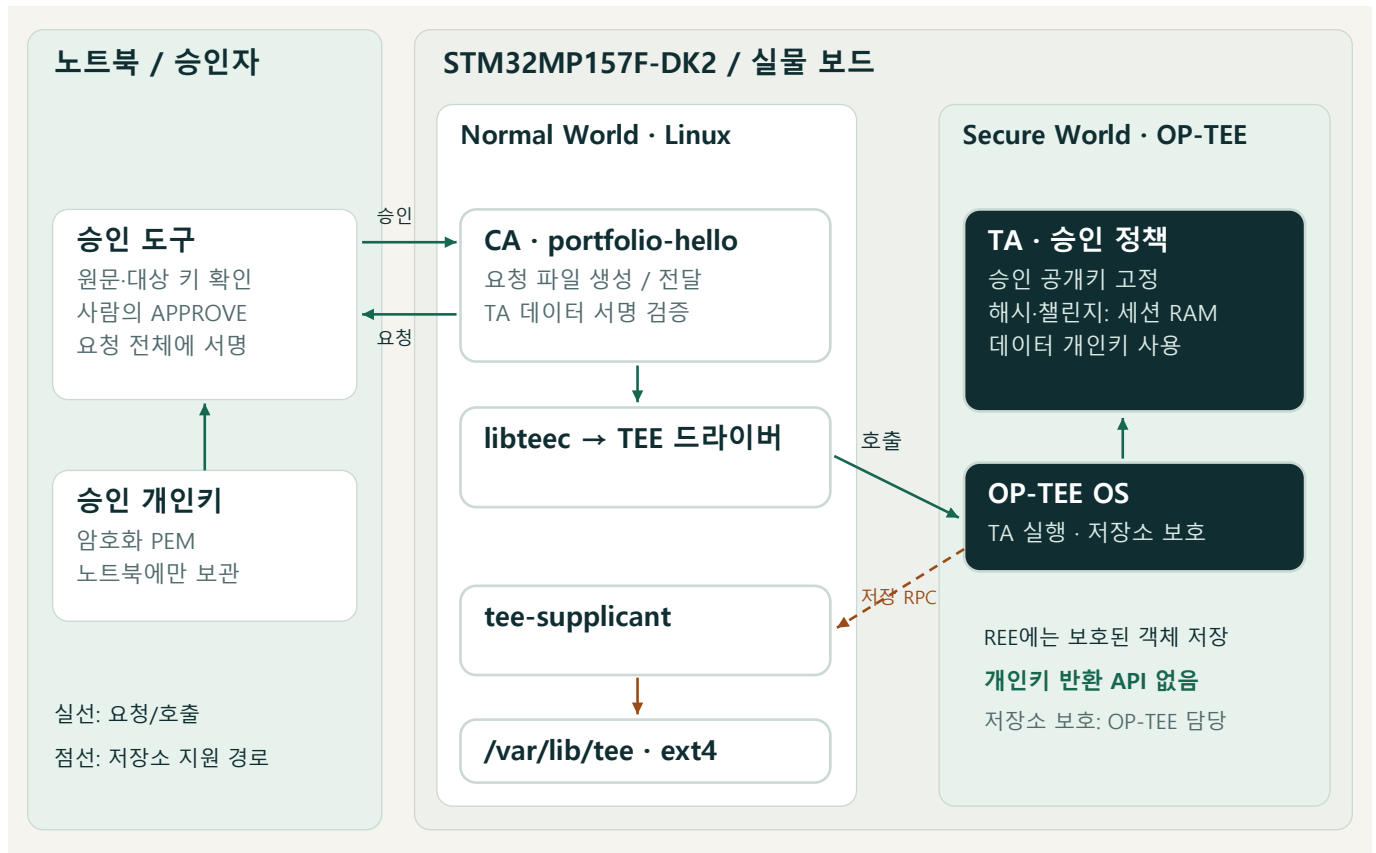
노트북 승인키는 요청 허가를 증명하고, TA 데이터 키는 허가된 데이터에 서명한다. 두 키를 분리했으며 개인키 반환 명령은 제공하지 않는다.

구현·통합 범위와 개발 방식

CA/TA 명령 계약과 세션 상태, 스트리밍 해시, 영속 서명 키, 132바이트 승인 요청 직렬화, 승인 검증·요청 소비, 노트북 승인 도구, Yocto 패키징 및 실물 시험.

초기 기능 구현 후 AI 검토·수정안을 diff로 확인해 반영하는 방식으로 진행했다. TA 입력 검사 보완에 도움을 받았다. OP-TEE 암호/저장 API, OpenSSL, ST BSP는 기반 기술로 사용했다. 본 자료는 실제 SSH 출력과 호스트 비교 결과를 요약하며, 화면 캡처를 합성한 자료가 아니다. 21 PASS는 상세 로드맵 전체 완료나 보안 인증을 의미하지 않는다.

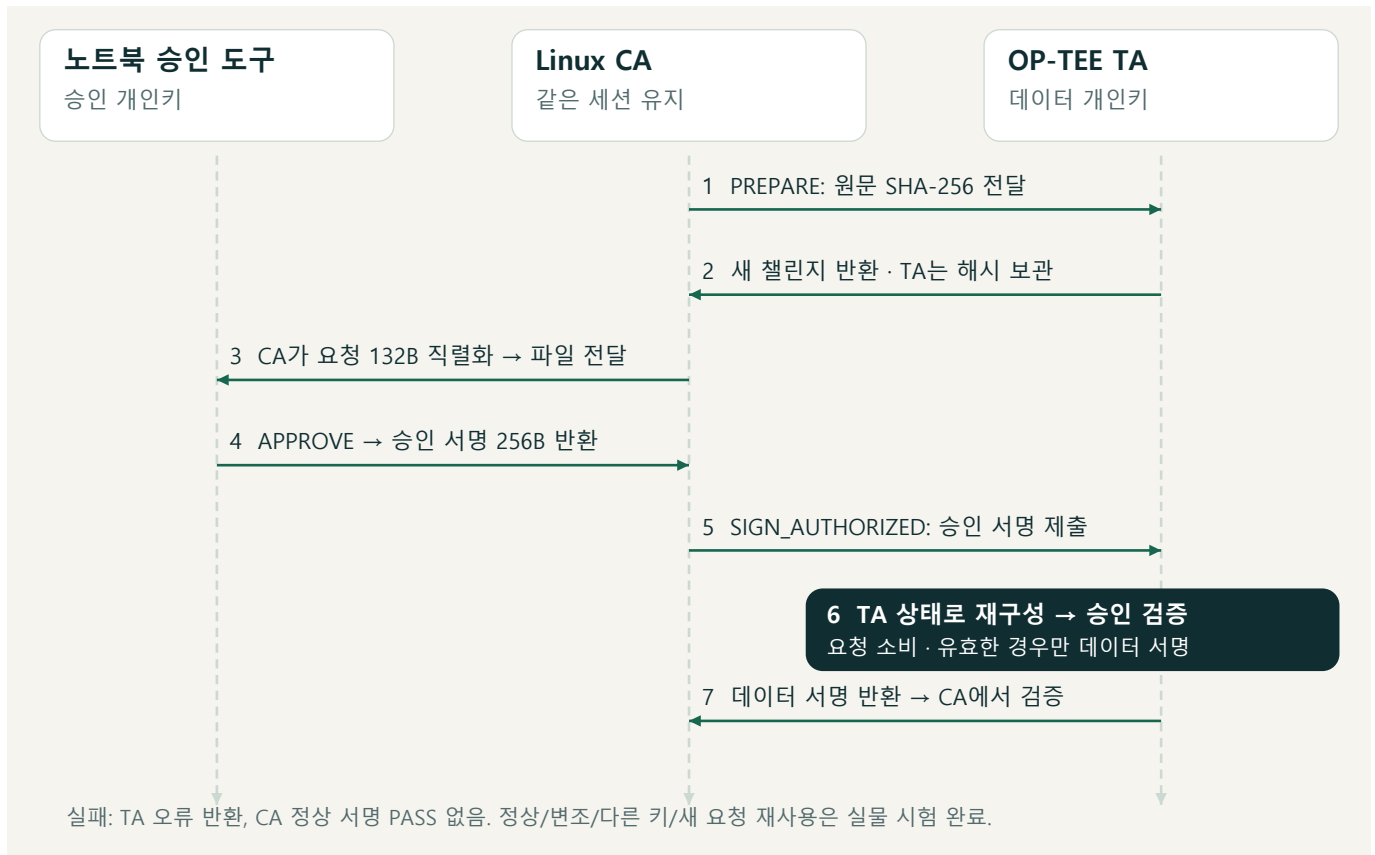
어디에서 실행되고, 어디를 신뢰하는가?



구성 요소	직접 맡은 역할
노트북 승인 도구	원문/대상 확인과 요청 서명. 승인 개인키는 노트북에 유지.
Linux CA / libteec	데이터 해시·요청 파일 준비, 세션 유지, 결과 서명 검증.
OP-TEE TA / OS	고정 승인 공개키로 검증, 실제 데이터 키 사용 및 영속 객체 관리.
tee-supplciant / REE FS	저장소 I/O 지원. Linux에 평문 개인키를 반환하는 API와는 다름.

화살표는 주요 기능 경로를 요약한다. 승인 파일 전달은 현재 수동 파일 전송이며 네트워크 승인 서버는 구현하지 않았다. Linux 침해나 저장소 롤백까지 방어했다고 의미하지 않는다.

하나의 요청을 승인하고 서명하기까지



같은 세션을 유지하는 이유

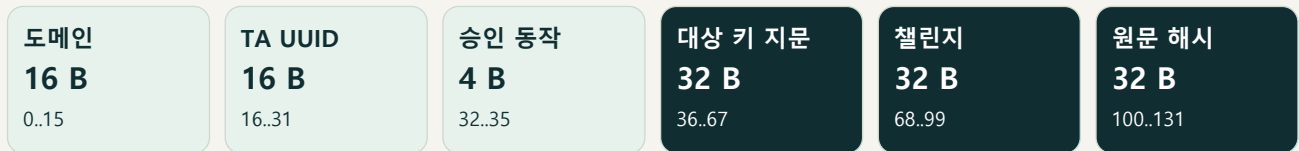
TA는 원문 해시, 랜덤 챌린지, 실제 데이터 키 핸들을 세션 상태에 연결한다. CA는 챌린지를 받은 뒤 공개키 지문과 해시를 조합하여 요청 파일을 만든다. 파일을 승인받는 동안 세션을 유지해야 같은 요청으로 검증된다.

서명 종류	검증 주체와 의미
노트북의 승인 서명	TA가 고정 승인 공개키로 검증: 이 요청에 대한 허가.
TA의 데이터 서명	CA가 데이터 공개키로 검증: 서명 대상 데이터의 일치.

실물 근거: 정상 승인은 성공했고, 승인 서명 변조·다른 승인키·새 요청에 과거 승인을 재사용한 경우 TA가 거부했다. 특정 Linux 프로세스 신원의 증명은 아니다.

요청 계약과 상태 수명

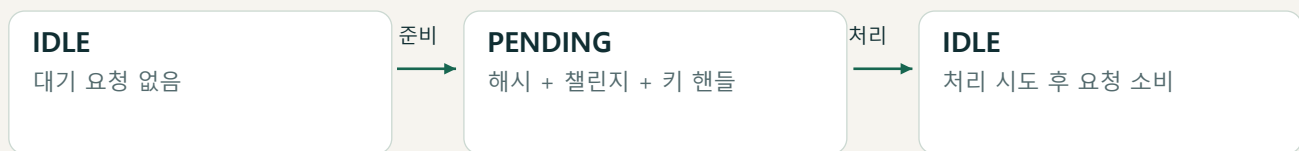
request.bin = 132 bytes



전체 132B → SHA-256 → 승인키로 RSA 서명 → approval.sig (256B)
TA는 공유 파일을 신뢰하지 않고 자체 세션·실제 키로 같은 바이트열을 재구성한다.

원문은 포함하지 않고 해시로 결합한다. 동작 8은 승인 대상 식별자이며 실제 제출 명령은 11이다.

RAM: 세션과 함께 사라지는 승인 상태



유효한 형식으로 처리 진입: 검증 실패도 요청 소비.
타입/길이/SHORT_BUFFER 사전 오류: 대기 요청 유지.

영속 객체: 세션을 닫거나 재부팅해도 같은 데이터 키를 다시 연다

세션 종료 → RAM 상태/키 핸들 정리

다음 세션 → 같은 TA UUID + Object ID로 기존 키 열기

실증: 재부팅 전후 공개키 동일. 키 객체와 세션 상태의 수명은 서로 다르다.

재사용을 거부하는 근거

메시지 해시뿐 아니라 대상 키·새 챌린지·TA UUID·동작을 함께 승인에 묶는다. 새 요청의 챌린지가 다르면 과거 서명은 TA가 재구성한 요청과 맞지 않는다. 키를 유지하는 것과 승인 상태를 유지하는 것은 별개다.

세션·버퍼·해시를 실물 보드에서 확인

최신 설치 패키지 하나로 기능을 회귀 시험했다. 과거 커밋으로 돌아가 기능별 시연을 구성하지 않았다. 보드와 로컬 deb의 CA/TA 해시도 일치했다.

시험	실제 확인 결과
PING 4회 → 새 실행 1회	같은 세션 1→4 누적, 새 실행은 다시 1
ECHO 일반 / 빈 입력 / 16바이트	입력과 반환 길이·내용 일치
잘못된 명령 / PING 타입	TA의 예상 오류를 확인하여 시험 PASS
SHA-256 분할 1 / 2 / 4바이트	모든 다이제스트가 sha256sum 기준값과 동일

실행 로그 발췌 — 카운터

```
TA : ping_cnt(1)
TA : ping_cnt(2)
TA : ping_cnt(3)
TA : ping_cnt(4)
PASS: 4 PING calls counted
TA : ping_cnt(1)
PASS: 1 PING calls counted
```

실행 로그 발췌 — SHA-256

```
$ portfolio-hello --hash test 1
$ portfolio-hello --hash test 2
$ portfolio-hello --hash test 4
sha256sum / TA outputs: same 32-byte digest
9f86d081884c7d659a2feaa0c55ad015
a3bf4f1b2b0b822cd15d6c15b0f00a08
```

위 명령의 각 실행은 exit=0이며 같은 결과를 반환했다. 다이제스트는 지면상 두 줄로 나뉘었다. 원본 로그에는 전체 64자리 값이 있다.

정상 승인 성공과 잘못된 승인 거부

TA가 보관한 메시지 해시와 챌린지를 기준으로 요청을 재구성한다. 노트북은 요청 전체에 서명하고, TA는 고정된 승인 공개키로 검증한 뒤 데이터 키를 사용한다.

입력 조건	TA 결과 / CA 종료	판정
정상 승인	0x00000000 / exit 0	PASS
승인 서명 1바이트 변조	0xffff3072 / exit 1	PASS
다른 승인키로 서명	0xffff3072 / exit 1	PASS
새 요청에 과거 승인 재사용	0xffff3072 / exit 1	PASS

실행 로그 발췌 — 정상 승인

```
SIGN_AUTHORIZED: result=0x00000000 origin=0x00000004
PASS: original data signature verified
PASS: altered data signature rejected
process exit=0
```

실행 로그 발췌 — 변조 승인

```
SIGN_AUTHORIZED: result=0xffff3072 origin=0x00000004
process exit=1
```

origin=0x00000004는 TA에서 반환된 결과다. 파일 전송이나 길이 검사에서 실패한 것을 승인 거부 성공으로 세지 않았다. 거부 시험에서 nonzero 종료는 기대 동작이므로 시험 판정은 PASS다.

시험 방법과 구분

정상/변조용 원본 서명은 사용자가 기존 도구에서 APPROVE와 키 암호를 입력했다. 다른 키 시험은 별도 메모리 내 시험키를 사용했고 기존 키를 교체하지 않았다. 재사용 시험은 성공했던 서명을 새 챌린지 요청에 제출했다.

재부팅 후에도 같은 키가 유지되는가?

재부팅 전 공개키를 호스트에 보존한 뒤 보드를 재부팅했다. 약 51.5초 후 새 Linux boot ID로 재접속하여 공개 키 n/e와 설치 파일을 다시 확인했다.

관측 항목	결과
Linux boot ID	재부팅 전후 다름: 실제 새 부팅 확인
공개키 modulus / exponent	호스트에서 전후 값을 직접 비교하여 동일
CA/TA 파일 SHA-256	재부팅 전후 및 로컬 deb와 동일
패키지 / 서비스	1.0-r0.10 유지 / tee-suppllicant active

동일 공개키 지문 — SHA256(n[256] || e[3])

```
BEFORE = AFTER
ffdaa92c3599bf82e9dd84d310dc7e43
e4d4a5ec6750a0d823150f2db5c9c0a5
Compared public n/e against pre-reboot capture: MATCH
```

지문은 두 줄로 표시했다. 단순 출력 유무가 아니라 실제 n/e 바이트를 비교했다. 재부팅 판정은 SSH 재접속과 boot ID 비교에 근거 하며 UART 부팅 영상은 아니다.

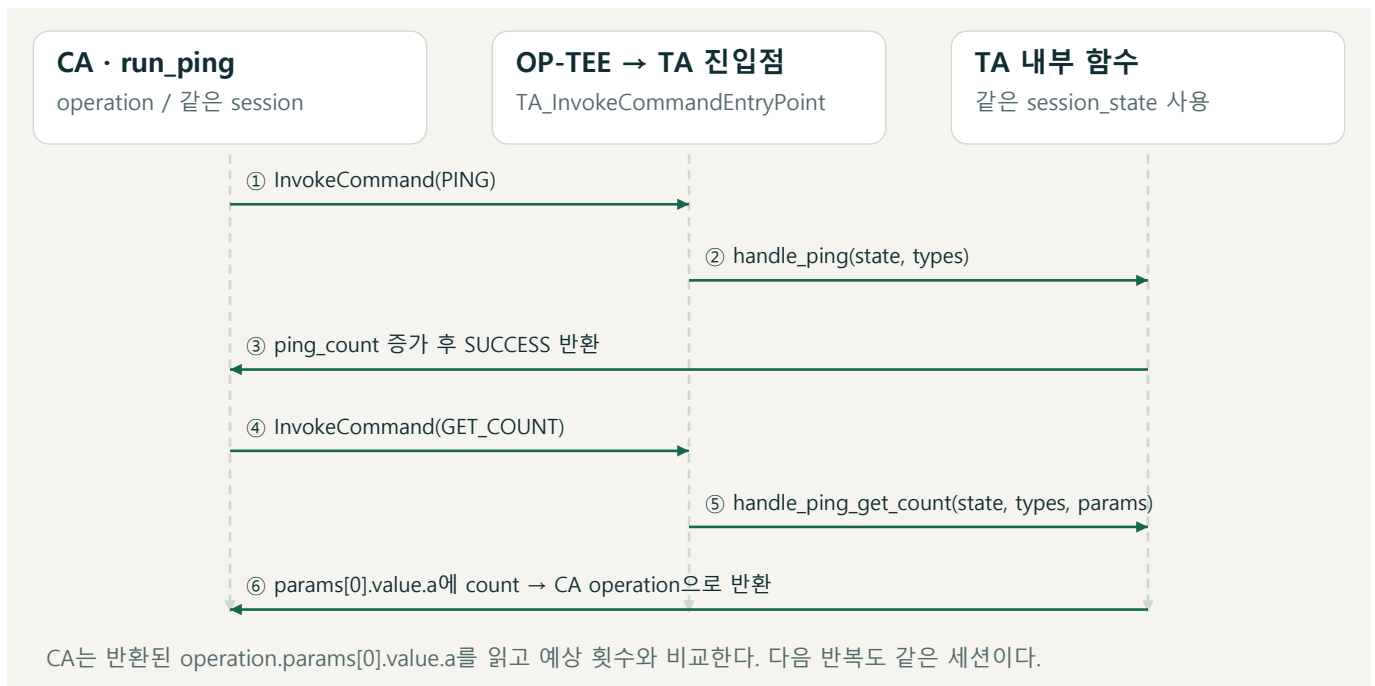
저장 위치와 보안 한계

기존 레시피 및 마운트 확인에서 REE FS 저장 위치는 /var/lib/tee이며 루트 ext4 파일시스템에 속한다. TA는 영 속 객체 API로 키를 저장하고 핸들만 세션에서 관리한다.

데이터보다 먼저 수명을 설계하기

문제 — 호출이 끝나도 남길 상태와 정리할 메모리는?

카운터와 해시 연산은 여러 요청 사이에 유지해야 한다. 반면 ECHO 호출의 버퍼를 TA가 소유한 메모리처럼 해제해서는 안 된다. 세션 상태와 호출 버퍼의 수명을 구분했다.



핵심 코드 — 세션 종료 시 소유한 자원을 정리

```

void TA_CloseSessionEntryPoint(void *session_context);

/* Body excerpt */
release_hash_operation(state);
clear_approval(state);
if (state->signing_key != TEE_HANDLE_NULL)
    TEE_CloseObject(state->signing_key);
TEE_Free(state);

```

TA_CloseSessionEntryPoint / ta/entry.c 91~95행. 함수 원형을 표시했으며, 본문의 앞선 NULL 검사는 생략했다. TEE_CloseObject는 열린 핸들을 닫으며 저장된 키를 삭제하지 않는다.

ECHO는 출력 용량이 부족하면 필요한 size를 반환한다. HASH_FINAL도 작은 버퍼 오류에서는 연산을 유지해 재시도를 허용하지만, 실제 최종화 시도 뒤에는 연산을 해제한다. 반복 BEGIN은 현재 계산을 보존하며 거부한다.

실물 확인 결과

PING 4회 누적·새 실행 1회 초기화, 일반/빈 입력/16바이트 ECHO, 1·2·4바이트 분할 해시의 sha256sum 일치를 확인했다.

웹 상세: 01 세션 → 02 버퍼 → 03 분할 SHA-256

발췌 코드 BSD-2-Clause / portfolio-hello contributors. 전문: assets/code/LICENSE.

키는 유지하고, 핸들은 닫는다

문제 — 재부팅 후 같은 키를 어떻게 다시 사용하는가?

핸들 주소를 저장하는 대신 키 속성을 영속 객체로 저장한다. 키 생성과 기존 키 열기를 별도 명령으로 분리하고, 생성에는 OVERWRITE를 넣지 않아 기존 객체를 보존한다.

객체 / 연산	수명과 역할
Transient RSA keypair	GenerateKey로 생성 → 영속 객체에 저장 → 임시 객체 해제.
Persistent object	같은 TA UUID / Object ID로 다음 세션에서 다시 열기.
Session signing_key	열린 객체의 핸들. 세션 종료 시 닫기.
Signing operation	키를 연결해 해시 서명. 연산 후 핸들 해제.

핵심 코드 — 기존 객체를 열고 실패를 확인

```
static TEE_Result open_signing_key(struct session_state *state);

/* Body excerpt */
/* Exclusive handle for this exercise; concurrent opens may conflict. */
result = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
    signing_key_obj_id, sizeof(signing_key_obj_id) - 1,
    TEE_DATA_FLAG_ACCESS_READ, &key);
if (result != TEE_SUCCESS)
    return result;
```

open_signing_key / ta/entry.c 158–163행. 다음 코드는 객체 종류·크기를 검사한 뒤에만 state에 핸들을 저장한다. 공유 플래그가 없어 동시 열기는 충돌할 수 있다.

개인키 보호와 공개키 검증을 구분

TA는 공개 속성 n/e를 내보내고 승인된 해시를 RSA PKCS#1 v1.5 / SHA-256으로 서명한다. CA는 OpenSSL 공개키 객체를 구성한다. public_check는 키 검사이고, verify는 서명·해시의 일치 검사다. 둘 다 공개키 소유자의 신원을 자동으로 증명하지 않는다.

실물 확인 결과

정상 승인 후 원본 서명 검증과 변경 데이터 거부, 재부팅 전후 공개키 n/e 동일성을 확인했다.

웹 상세: 04 키 생성·서명 → 05 영속 저장

발췌 코드 BSD-2-Clause / portfolio-hello contributors. 전문: assets/code/LICENSE.

승인된 데이터만 서명하기

문제 — 승인받은 해시 A 대신 해시 B를 보내면?

CA가 승인을 받은 뒤 다른 해시를 보내고, TA가 그 값을 그대로 서명한다면 승인 대상과 서명 대상이 달라진다. CA의 호출 순서에 기대지 않고 TA가 둘을 연결해야 한다.

핵심 코드 — 세션의 해시를 보존하고, 승인 후 서명

```
static TEE_Result handle_sign_authorized(struct session_state *state,
                                         uint32_t types, TEE_Param params[4]);

/* Body excerpt */
TEE_MemMove(message_digest, state->pending_digest, sizeof(message_digest));
result = build_pending_request(state, request);
// ... request consumption / hashing / error checks omitted ...
result = verify_approval(request_digest, approval);
if (result != TEE_SUCCESS)
    goto cleanup;
output_size = TA_PORTFOLIO_RSA_BYTES;
result = sign_stored_digest(state, message_digest,
                             params[1].memref.buffer, &output_size);
```

handle_sign_authorized() / entry.c 380–381, 391–395행 비연속 발췌. 중간 처리와 앞뒤 입력 검사·정리는 생략하고 긴 호출만 줄바꿈했다. 실행 가능한 전체 함수는 아니다.

설계 이유 — 승인한 요청과 실제 키 사용을 연결

- ① **대상 고정**: pending_digest는 처음 요청에서 TA가 보관한 해시다.
- ② **요청 결합**: build_pending_request()가 세션의 해시·챌린지·실제 키로 요청을 재구성한다.
- ③ **승인 강제**: verify_approval() 실패 시 서명 경로에 도달하지 않는다. 성공하면 보존한 같은 해시를 sign_stored_digest()에 전달한다.

생략한 구간은 요청을 소비하고 재구성한 요청의 해시를 계산한다. 사전 파라미터 검사 이후의 처리 시도는 실패해도 소비한다. 챌린지는 CA 신원 인증이 아닌 특정 요청의 승인 검증에 사용한다.

실물 보드에서 확인한 결과

정상 승인	변조 · 다른 승인키 · 새 요청에 과거 승인
서명 및 원본 데이터 검증 성공	TA 거부 / 예상 오류와 종료값 확인

2026-10-02 실증.

상세 코드 설명 ↗ 입력 검사 · 협업 과정 · Git 이력 · 기능별 코드 위치

발췌 코드: BSD-2-Clause / portfolio-hello contributors. 라이선스 전문: 공개 저장소 assets/code/LICENSE 및 웹 설명.

실행 근거와 한계를 함께 제출

실증 회차: p0-20261002T115454Z

설치 패키지: portfolio-hello_1.0-r0.10_armhf.deb

deb SHA-256

```
3a40219abc43714e549975a67e2c5402
9eca297309493dc1660e832d1d7feeea
```

조사 시 저장소 HEAD

```
44b7917a028a1eb6863b4bdc7b68ec05a9abff0f
```

보드 CA/TA 파일과 로컬 deb의 일치는 확인했다. 위 HEAD가 해당 deb의 과거 빌드 입력이었다는 증명은 확보하지 않았으므로 빌드 커밋으로 단정하지 않는다. 이번 실증 중 빌드·패키지 재설치·전체 이미지 재기록은 수행하지 않았다.

제출 자료 연결

GitHub: [grapesgun67/stm32mp1-optee-showcase](https://github.com/grapesgun67/stm32mp1-optee-showcase)

저장소 내 증거: [evidence/p0-20261002T115454Z/report.md](#)

사례 문서: [docs/case-study.md](#)

웹페이지: [index.html](#)

GitHub Pages 사이트 주소

<https://grapesgun67.github.io/stm32mp1-optee-showcase/>

공개 저장소에는 실증 문서·로그·PDF를 제공한다. 전체 CA/TA 소스와 개발 이력은 별도 비공개로 관리하며, 검토한 코드 일부와 설명은 공개한다. 공개 URL을 제출하기 전 Pages 배포 성공과 실제 접근을 확인한다.

개발 방식과 확인 범위

초기 구현·커밋 → AI 검토 → diff 확인 → 선택 반영. TA 타입·버퍼·값 검사 보완에 AI 도움을 받았다. 개별 줄의 작성 기여를 단정하지 않는다.

동시 세션·카운터 경계, ECHO 내장 NUL·작은 출력 버퍼 재시도, HASH 오류 순서, 동일 세션 재전송·직접 명령 우회, 이전 공개키 독립 검증 및 키 교체·회수는 후속 검증 항목이다. REE FS 단조 카운터 경고는 미해결이며 HUK·보안 부팅·롤백 방지는 입증하지 않았다. 실제 차량 적용, AUTOSAR/Uptane 준수 또는 제품 수준 보안 인증을 주장하지 않는다.

다음 확장은 A7↔M4 및 외부 MCU 통신, 메시지 인증, OTA 설치·복구다. 현재 입증한 키 관리와 요청 승인 범위를 출발점으로 삼는다.